

WeUP Event Platform – Technical Requirements Document (TRD)

Introduction

WeUP is a next-generation event platform that combines **immersive 3D venue maps**, **live in-event shops**, and a **tiered, invite-only user community** into a unified experience. This Technical Requirements Document (TRD) provides a comprehensive technical overview of WeUP's architecture and components, intended for internal engineering teams and external integration partners. The document balances clarity with technical depth – covering both backend and frontend aspects – to ensure all stakeholders (from software developers to venue partners) share a common understanding of the system's design and requirements. Key focus areas include the 3D mapping stack, real-time features, e-commerce integration, geolocation and indoor positioning, data architecture, microservices design, security, and forward-looking extensions. In the sections that follow, each major component of the WeUP platform is detailed with its architecture, technology stack, and design rationale. The goal is to outline how these pieces come together to deliver a **scalable, real-time**, and **immersive** event experience that can evolve with future innovations.

Platform Overview

WeUP's platform comprises several interrelated features: an interactive 3D map of event venues, a real-time "live shop" for on-site commerce, a gamified community system, and a suite of supporting backend services. At a high level:

- **Immersive 3D Venue Maps:** Users can explore event locations (conference halls, festival grounds, etc.) via a high-fidelity 3D map. The map supports seamless transitions from outdoor environments (e.g. city or campus view) to detailed indoor floorplans. It also displays dynamic overlays like crowd density heatmaps and booth/shop statuses in real time.
- **Live In-Event Shops:** Vendors and organizers can set up virtual shops tied to physical booths. Attendees browse items on the app, see real-time inventory, and make purchases for pickup or delivery. The system ensures inventory counts are synchronized live and handles payments securely within the app.
- **Tiered Invite-Only Community:** WeUP is an invite-gated platform, where new users join via invitations from existing members or event organizers. Users belong to different **tiers** (e.g. general member, VIP, organizer) which grant varying access levels and perks. The community features social graph elements (friend connections, follower/fan relationships) and gamification (points, badges) to encourage engagement.
- **Gamification and Engagement:** The interface includes game-like elements – e.g. quests or scavenger hunts at events, leaderboards for participation, and reward points for actions like visiting booths or inviting new members. This not only enhances user engagement but also drives the organic growth of the invite-only network.
- **Backend Microservices Ecosystem:** The platform's backend is composed of modular **.NET 8 microservices**, each responsible for specific domains – examples include mapping service, shopping

service, ticketing, payments, user management, gamification, and real-time communication. These services communicate via REST/gRPC and asynchronous events, persisting data primarily in **PostgreSQL** (for relational data) and **Redis** (for caching and fast lookups).

- **Real-Time Communication:** Live data – such as map updates (crowd movement, availability changes) or flash sale notifications – is pushed to clients through a real-time messaging infrastructure (WebSockets/SignalR). This ensures that the user's view is always up-to-date without manual refresh.
- **Security and Access Control:** The platform enforces robust authentication (with support for OAuth2/JWT tokens) and role-based authorization. The invite model and tiered access are implemented at the identity service level, gating features and content based on the user's verified tier. Additional measures like two-factor authentication and device linking are considered for sensitive actions (e.g. high-value purchases).

Together, these components enable WeUP to offer an **interactive, data-rich, and secure** event experience. In the next sections, we delve into each technical area, starting with the 3D mapping stack that forms the core of the user's immersive interaction.

System Architecture Overview

High-level architecture of the WeUP platform, showing frontend clients, the API gateway, and major backend microservices (mapping, shop, ticketing, etc.), all communicating through secured channels. The architecture emphasizes modular services and real-time data flow.

At a high level, WeUP's system architecture follows a **client-cloud microservices model**. The **frontend** consists of a web Single-Page Application (SPA) (built with Vue 3 and Quasar) and companion mobile apps, which connect to backend services via a unified API layer. The **backend** is composed of multiple containerized microservices developed in .NET 8, each focusing on a specific business domain. Key elements of the architecture include:

- **API Gateway:** A gateway (e.g. using Ocelot or Azure API Management) serves as the single entry point for all client requests. It routes API calls to the appropriate microservice and handles cross-cutting concerns such as authentication, rate limiting, and response aggregation. This follows the API Gateway pattern to simplify client interactions with numerous services ¹.
- **Microservices:** Independent services encapsulate functionality like Map Service, Shop Service, Ticketing, Payments, User/Identity, Gamification, Real-Time Updates, etc. Each service owns its **own data store** (autonomy), typically a schema in PostgreSQL or a dedicated database instance. This autonomy aligns with microservice best practices ², ensuring each service can scale or evolve independently.
- **Database and Caching:** **PostgreSQL** is the primary relational database for persistent data (users, events, orders, tickets, etc.), chosen for its reliability and spatial data support (PostGIS extension can aid geolocation features). **Redis** is used for caching and fast transient data access – e.g. caching map tile metadata, session tokens, and frequently accessed data to reduce load on databases. Redis also acts as a **distributed cache** to share state across instances (and can serve as a pub/sub broker for certain real-time features).
- **Communication Pattern:** Services communicate **synchronously** via REST/gRPC calls for request-response needs (for example, the Shop Service calling Payment Service during checkout) and **asynchronously** via an event bus (like RabbitMQ or Azure Service Bus) for event-driven updates (e.g.

an OrderPlaced event to update inventory, trigger email confirmation, etc.). This combination allows loose coupling and resilience: non-critical updates propagate through events without blocking user-facing operations ³.

- **Real-Time Messaging:** A dedicated real-time subsystem (using WebSocket technology via .NET's SignalR or similar) pushes live updates to clients. This may be integrated as part of each service (e.g. Map Service broadcasting location changes) or as a centralized **Real-Time Service** that other microservices publish updates to, and which then fans out to connected clients. SignalR in ASP.NET Core provides a simple programming model to push server-side events to clients in real-time ⁴, which is essential for live dashboards and maps.
- **Security & Auth:** An **Identity Service** handles user authentication and authorization. It issues JWT tokens to clients upon login/invite acceptance, and the API gateway and microservices validate these tokens. Role-based access control is enforced according to user tier (e.g. VIP vs standard user vs admin) and event permissions. The invite-only logic is also handled here: new accounts must present a valid invite code or link associated with an existing member.
- **Deployment & Scaling:** All services are containerized (e.g. Docker) and orchestrated in a cloud environment (Kubernetes or Azure Container Apps, etc.). Services can scale horizontally based on load – for instance, the Map Tile service might scale out during a large event to handle many concurrent 3D tile requests, and the Real-Time service will scale to handle WebSocket connections. The stateless nature of microservices (with state offloaded to DB/Redis) facilitates scaling.

This modular architecture ensures that the WeUP platform is **scalable, resilient, and maintainable**. Each microservice can be developed and deployed independently, enabling parallel development and continuous deployment without affecting other services ⁵ ⁶. In the following sections, we explore each major subsystem in detail, starting with the 3D mapping stack.

3D Map Stack and Architecture

Example of a 3D venue map rendered with CesiumJS, showcasing high-rise buildings and terrain. WeUP's map stack will deliver similar high-fidelity 3D views, both outdoor and indoor, with smooth performance.

The 3D map is a cornerstone of WeUP, providing users with an interactive “digital twin” of the event venue. The stack leverages **WebGL-based mapping engines** – primarily **CesiumJS** for full 3D globes and models, potentially complemented by **Mapbox GL JS** for base maps and 2D/2.5D visualizations. The architecture of the map system is designed to handle **outdoor-to-indoor transitions**, real-time overlay of dynamic data, and efficient streaming of large 3D assets:

- **CesiumJS for 3D Streaming:** CesiumJS offers a high-performance 3D engine capable of rendering global-scale and high-detail environments in the browser ⁷. It uses the open **3D Tiles** standard for streaming massive heterogeneous 3D datasets (like photogrammetry, BIM models, point clouds) with level-of-detail control ⁸ ⁹. WeUP will utilize Cesium for rendering detailed indoor venue models (e.g. a convention center interior) and outdoor terrain. By using 3D Tiles, the client only loads the necessary data for the current view/zoom, enabling smooth visualization even for very large venues or campuses. For example, a venue scanned into a multi-gigabyte point cloud can be served via 3D Tiles so that only an approximation is loaded at first, with finer details loading as the user zooms in – achieving real-time rendering of terabyte-scale data ⁹.
- **Mapbox GL for Basemap and 2D Layers:** In addition to Cesium, the platform may integrate Mapbox for high-quality base maps and 2D mapping features (such as street maps for the area around the

venue, or floorplan-style views). **Mapbox's indoor mapping capabilities** (as demonstrated by the MapsIndoors platform) can blend indoor and outdoor use cases seamlessly ¹⁰. This means a user could zoom out to see the event location in the context of its city (using Mapbox's vector tiles and terrain), then zoom in and transition into a fully detailed indoor view (handed off to Cesium with a 3D model of the venue). Both engines support high-fidelity rendering and smooth performance on modern browsers ¹¹ ¹². WeUP will choose the best tool per layer: for instance, using Mapbox for fast 2D navigation and labels, and Cesium when true 3D visualization or global context is needed.

- **Indoor/Outdoor Transition:** A critical aspect is the user experience moving between outdoor maps and indoor venue models. WeUP's design will likely use an **automatic transition trigger** – e.g. when the user zooms into a building or selects an "Enter Venue" button, the app switches from Mapbox's map to loading the detailed indoor model via Cesium. The system can smoothly interpolate this by aligning coordinate systems (both Cesium and Mapbox use latitude/longitude; the indoor model can be geo-referenced to the same coordinates). Mapbox's vector maps provide context at city or campus scale, and once a building is entered, Cesium's 3D model provides the interior detail. From the user's perspective, this should feel like a continuous zoom rather than a jarring switch.
- **Dynamic Overlays:** The 3D map will display various real-time overlays:
 - *Crowd Flow Heatmaps:* Aggregated positioning data (from users' mobile devices or venue sensors) can be visualized as heatmaps or density contours on the map. For example, if one hall is crowded, the floor area on the map might glow in red. This overlay updates in real time, requiring efficient data push and rendering updates (possibly using Cesium's entity API or custom WebGL layers).
 - *Booth/Shop Status:* Each vendor booth or session area on the map is interactive. The status (e.g. **"Line length"**, **"Out of stock"**, or **"Now Presenting"** for a stage) can be indicated with colored markers or icons above the 3D location. These overlays are updated via real-time events from the backend (e.g. Shop Service sends an update when an item sells out, triggering the map to mark that booth). The mapping engine supports dynamic entity updates – Cesium's **Dynamic Scene Layer** allows time-dynamic data and entity tracking ¹³, which we will utilize for animating movements or status changes.
 - *User & Friend Positions:* Where privacy and permissions allow, the map can show the live position of the user themselves (blue dot similar to GPS in maps) and possibly their friends or party members in the venue (with consent). This involves continuously updating those positions via the geolocation service and pushing updates to the map. Because direct real-time sharing of all users' locations can raise privacy concerns, WeUP may restrict this to friend groups or use coarse-grained crowd visualization for general users.
- **Real-Time Map Updates:** All overlay data on the map is driven by real-time events. WeUP will use WebSockets to push updates to the front-end map view immediately. Modern users expect up-to-date info without refresh ⁴ – for instance, if an item goes out of stock or a session changes, the map indicator updates within seconds. The map component will subscribe to relevant channels (e.g. a channel for crowd heatmap updates, one for shop updates) and apply updates by modifying the in-memory map entities. Performance is crucial: updates will be diff-based (only changes are sent, not the entire dataset each time).
- **Performance and Level of Detail:** To ensure smooth rendering on a variety of devices (from high-end desktops to mobile phones), the map stack will use level-of-detail (LOD) techniques:
 - The Cesium 3D Tiles already handle LOD for geometry and textures (less detail when far away, more when close). We will generate or obtain venue models that include multiple LOD layers, possibly simplifying interior details at distance.
 - On the application side, we will limit the frequency of certain updates (e.g. crowd heatmap might update every few seconds rather than continuously streaming each person's movement) to avoid overwhelming the client.

- We'll also leverage hardware acceleration fully; both Mapbox GL and Cesium are GPU-intensive but tuned for efficiency – Cesium's engine manages **planetary-scale data with millimeter precision** by optimizing WebGL usage ¹⁴ ¹⁵ . WeUP will adhere to recommended best practices (like batching entity updates and avoiding excessive DOM interactions) to maintain high frame rates even with many data overlays.

In summary, WeUP's 3D Map subsystem will combine the strengths of Cesium and Mapbox to deliver an **immersive and responsive mapping experience**. Users can navigate from a broad overview down to a specific booth inside a hall, all in one continuous interface. Under the hood, efficient 3D streaming and real-time data integration ensure that what users see accurately reflects the current state of the live event.

Live Shop Infrastructure

One of WeUP's standout features is the **Live Shop**, which enables on-site commerce tightly integrated with the event experience. This ranges from merchandise booths selling products to food vendors or artists selling digital goods. The technical requirements for the shop infrastructure revolve around **real-time inventory synchronization**, **smooth purchasing workflows**, and **transaction integrity** in a high-concurrency environment. Key design points include:

- **Microservice Design – Shop and Inventory Services:** The e-commerce functionality is split into dedicated services. The **Shop Service** manages products, pricing, and shop metadata (e.g. which event and booth a shop is associated with). The **Inventory Service** (or an inventory module within Shop) keeps track of stock levels for each item in real time across all users. By separating inventory logic, we can ensure that inventory updates (which are frequent during an event) are lightweight and scalable. The Inventory Service will provide APIs to query stock and to reserve or decrement stock when orders are placed. It must enforce concurrency control so that two users can't purchase the last item simultaneously. Techniques like **optimistic concurrency control or distributed locking** on inventory records will be used to prevent inconsistencies ¹⁶ . For example, using a row version number in PostgreSQL and updating stock with `WHERE stock > 0 AND version = X` ensures only one transaction succeeds in grabbing the last unit, or employing a Redis-based lock for the item during the purchase operation.
- **Real-Time Inventory Synchronization:** When an item is purchased or restocked, that change should propagate instantly to all viewing users. If a popular limited-edition item has only 1 unit left, once it's gone the UI needs to update (perhaps the "Sold Out" label appears). WeUP uses a publish/subscribe model here: the Inventory Service publishes an event (e.g. `ItemSoldOut` or `InventoryUpdated`) which is delivered via WebSocket to clients viewing that item or shop. Because of the volume of potential shops and items, a naive approach of broadcasting every update to every client is inefficient; instead, the frontend will subscribe to updates for the specific shops/items in view (for example, when a user opens a booth's page, the client subscribes to that booth's channel for inventory changes). The Inventory Service can also leverage Redis's **in-memory speed** for hot data – caching inventory counts in Redis so that frequent reads (like many users checking an item's availability) are fast ¹⁷ . This cache is kept in sync via writes through to the DB and cache invalidation on changes.
- **Commerce Workflow & Payment Integration:** The typical purchase workflow in WeUP involves multiple steps: browse items -> add to cart -> initiate checkout -> payment -> confirmation. The **Payment Service** will integrate with external payment gateways (such as Stripe, PayPal, or in-app payment SDKs). During checkout, the system must securely handle payment data (likely via

tokenization or redirecting to the payment provider's UI to avoid direct handling of sensitive card data). WeUP ensures **transactional integrity** by coordinating inventory reservation with payment:

- When a user checks out, the Shop/Order Service requests the Inventory Service to **reserve** the items (mark them as in-cart or reduce available count) before payment is processed. This reservation might last a short time-window.
- If payment succeeds, the order is finalized and inventory is permanently decremented. If payment fails or times out, the reservation is released (stock is returned to available pool).
This pattern prevents scenarios where an item is sold to multiple people or remains stuck in someone's cart unpaid. Using robust ACID transactions in PostgreSQL can help here – e.g. a stored procedure that decrements stock and logs an order in one transaction, or using a saga pattern across microservices where the Payment Service and Inventory Service communicate to confirm or rollback the reservation.
- **Order Service:** A separate **Order Service** will handle order records, status, and workflow (e.g. "Pending Payment", "Completed", "Ready for Pickup"). It orchestrates between Inventory and Payment. For instance, it might create an order record as pending, call Inventory to reserve items, call Payment gateway for charge, then update order status accordingly. The Order Service must also handle edge cases: partial failures, refunds, cancellations (perhaps an attendee cancels an order, or an item becomes unavailable – these trigger restocking or refund flows).
- **Data Integrity & Consistency:** Given that events can have patchy connectivity and high load, the system must be resilient. WeUP will implement measures like:
- **Idempotent operations:** Ensuring that if a client accidentally submits a purchase twice (e.g. double click), it doesn't double-sell the same item. Each order or payment transaction gets a unique ID to prevent duplicates.
- **Transaction logging (Outbox pattern):** Important changes (orders placed, payments completed) are logged in a durable store or message outbox so that if a service crashes mid-process, a background worker can resume and complete the work or at least not lose the event.
- **Monitoring and Alerts:** The Shop system will include monitoring for inventory levels (to alert vendors when stock is low) and failed payment attempts or other anomalies. This ensures issues can be resolved promptly during a live event.
- **Scalability for High Concurrency:** Live events, especially product launches or flash sales, can cause surges of activity. WeUP's shop architecture is designed to **scale horizontally** – multiple instances of the Shop/Inventory/Order services can run concurrently. A load balancer or service mesh will distribute requests. Using **PostgreSQL** provides strong consistency for inventory updates, but we will carefully tune it (proper indexing, perhaps partitioning by event) to handle high write throughput. In cases of extremely high scale (tens of thousands of simultaneous buyers), an alternative like an **event-sourced inventory** with a log (Kafka) and eventual consistency might be considered, but initially the simpler relational model with transactions will be used since it's easier to reason about for correctness. If needed, we could incorporate a message broker to queue order requests during a surge, processing them sequentially – this effectively creates a **queue system for flash sales** to prevent overload and to maintain fairness (the queue ensures orders are handled in order of arrival).
- **User Experience Considerations:** The app's UI will reflect inventory status in real-time, and handle edge cases gracefully. For example, if two users try to buy the last item at the same time, one will succeed and the other's payment will not be processed and they'll receive an apology/out-of-stock message. Ensuring that this case is handled smoothly is important to user trust. Additionally, the Live Shop can support features like **wish lists or notify-me** if something comes back in stock, which can be implemented by subscribing users to events for that product.

Overall, the Live Shop infrastructure merges the reliability of traditional e-commerce systems with the immediacy of real-time event tech. By doing so, it creates a seamless experience: attendees can confidently shop during events, and vendors have an efficient way to reach customers without separate systems.

Geolocation and Positioning

WeUP bridges the digital and physical worlds of an event, so precise geolocation is vital. The platform will employ a hybrid approach to determine where users are in and around venues, leveraging **GPS for outdoor positioning** and **Bluetooth Low Energy (BLE) beacons and other sensors for indoor positioning**. In advanced scenarios, device-based **LiDAR and Visual Inertial tracking (ARKit/ARCore)** can further enhance indoor accuracy. Key components and strategies for positioning include:

- **Outdoor Positioning (GPS):** When users are outdoors (e.g. approaching a venue or in an open festival ground), standard GPS from their smartphones provides location. GPS gives accuracy of about 5-10 meters in good conditions, which is sufficient to locate which building or area a user is in. The WeUP app will use the HTML5 Geolocation API (or native app location services) to get coordinates when permission is granted. However, GPS accuracy degrades indoors and in dense urban environments (multipath, signal loss), so while GPS might indicate which building the user is in, it's **not enough for precise indoor navigation** ¹⁸. WeUP will use GPS primarily to identify the initial context (e.g. user has arrived at the venue) and perhaps to place the user's dot on the outdoor map correctly. Once a user enters a venue, the system transitions to indoor methods.
- **Indoor Positioning with BLE Beacons:** WeUP will support deployment of **BLE beacon** infrastructure inside venues to provide better indoor localization. BLE beacons (like iBeacon or Eddystone) are small transmitters placed at known fixed locations (e.g. pillars of a convention center or specific booths) that broadcast signals. The mobile app can detect these signals – by measuring the signal strength from multiple beacons, the app can approximate the user's position via triangulation or fingerprinting. This approach is a **proven, mature technology** for indoor navigation, often achieving ~5m accuracy ¹⁹ ²⁰. WeUP will work with event organizers to install and map these beacons: the beacon IDs and their coordinates must be stored in the Map/Location Service. The app, upon sensing beacons, will send the data to our positioning algorithm (possibly running on-device or cloud) to compute the location. We might employ a **sensor fusion** approach – combining beacon data with phone sensors (accelerometer, compass, etc.) for smoothing. Beacon-based navigation doesn't rely on visual input, so it's robust even when a camera can't be used (like in dark or very crowded areas) ¹⁹.
- **LiDAR and AR-Based Positioning:** Many modern smartphones (iPhones with ARKit, Androids with ARCore) can use their cameras and sensors to do **Visual-Inertial Odometry** and even build local maps. Apple's ARKit, in particular, can use the built-in LiDAR scanner (on Pro models) to create a depth map of the environment. By using **ARKit's localization anchors** and LiDAR, the app can recognize a known environment and precisely locate the device within it ²¹. For example, if the venue's 3D model is loaded into the app, the AR module might match what the camera sees (e.g. the shape of a hallway or a specific marker) to the model, thus "anchoring" the user's position with high accuracy (centimeter-level in some cases). This essentially is an example of a **Visual Positioning System (VPS)** – using the camera to identify where the user is by recognizing the surroundings ²². WeUP can incorporate an AR mode where a user points their phone and the app shows AR overlays (directions, booth info) on the camera feed. In that mode, ARKit/ARCore are doing the tracking. While ARKit is iOS-only, ARCore covers Android; both can be leveraged to improve indoor nav (keeping in mind ARKit's LiDAR capability gives it an edge in depth sensing).

- **Hybrid Positioning Algorithm:** The platform will likely use a combination of methods to improve reliability:
- **Sensor Fusion:** As suggested, combining readings from GPS, Wi-Fi, BLE, accelerometer, gyroscope, etc., leads to more stable results ²³ ²⁴ . For instance, a user walking inside could be tracked by dead-reckoning (using accelerometer steps and gyroscope turns) between known anchor points (like last BLE fix). The system can also use known constraints (you can't walk through walls – the indoor map graph can constrain possible paths).
- **Kalman filters or Particle filters** might be used under the hood to continuously estimate the user's position given multiple sensor inputs.
- **Wi-Fi Positioning:** WeUP can also optionally use Wi-Fi signals if available – either via fingerprinting (mapping signal strengths in various spots) ²⁵ or newer RTT (round trip time) ranging if hardware supports. Many venues already have Wi-Fi, so this is an extra signal source for fusion.
- **Integration with Map and Floor Plans:** The geolocation coordinates (latitude/longitude or local XYZ in a building coordinate system) need to map onto our 3D venue model. For outdoor, that's straightforward (GPS lat/long places the user's dot on the map). For indoor, WeUP will maintain a coordinate mapping for each venue. If the venue 3D model is georeferenced (i.e. aligned with real world coordinates), then the indoor positions can be translated to lat/long (many indoor mapping solutions use the same global coordinate space, just with altitude or special floor identifiers). Alternatively, we might treat each building with its own local coordinate system (like 2D coordinates on a floor plan plus a floor level index); then we must convert the user's sensor readings to that local system. BLE beacon approach inherently gives positions in the venue's coordinate system (because we program the beacon coordinates). AR-based approach can output pose relative to the venue model if the model is loaded.
- **Multi-floor handling:** If a venue has multiple levels, the positioning system must also detect floor transitions. Barometric pressure sensors (in phones) can assist by detecting altitude changes (floor changes often ~3-5 m difference in pressure). Also, if AR is used, recognizing distinct features on different floors could tell which level the user is on. WeUP will incorporate floor detection logic so it shows the correct level map.
- **User Privacy and Controls:** Tracking a user's location continuously raises privacy considerations. WeUP will be transparent and opt-in: users must grant location permission, and possibly explicitly enable "indoor navigation mode" when they want to use it. For sharing with friends, explicit consent is needed (e.g. "Share my location with my friends at this event"). WeUP's system will anonymize any data used for crowd heatmaps – we only need aggregate densities, not identifiable info, for the crowd overlay. Precautions will be taken that any location data sent to the backend (for analytics or heatmaps) is handled according to privacy laws (e.g. not stored beyond what's needed, possibly anonymized or aggregated).
- **BLE Beacon Management:** The platform needs tools for venue operators to map and maintain beacons. Likely an admin interface allows input of beacon UUIDs and their exact coordinates (and maybe height). If a beacon fails or is moved, the system should be updated. WeUP might also support **temporary beacons** for events that are not permanently instrumented (e.g. provide a kit to set up a few reference beacons).
- **Performance:** Frequent location updates can be battery-intensive on mobile devices. WeUP's mobile app will use efficient background processing – for example, using significant location change updates rather than constant GPS polling when possible, and using BLE scans judiciously. AR mode will naturally use more battery, so it will likely be a user-invoked mode. The system will also throttle server-side updates for moving users (no need to send every single footstep to the server; local smoothing can be done and server updated maybe once every few seconds or when significant movement occurs).

By combining GPS, BLE, and advanced AR techniques, WeUP aims to provide **reliable indoor positioning** – a notoriously hard problem – in a way that feels seamless to users. Users can find their way to a booth or friend in a crowd through the app’s guidance, enhancing the on-site experience. And beyond navigation, this positioning data feeds into analytics (like heatmaps, popular areas) which can be invaluable to event organizers.

LiDAR and Point Cloud Processing Pipeline

To create the rich 3D venue maps used in WeUP, we need to generate 3D models of event spaces. This may involve **LiDAR scanning** or photogrammetry to capture real-world venues and converting that data into a format suitable for real-time rendering. The pipeline from raw spatial data to an interactive 3D map involves several stages:

- **Data Acquisition (Scanning):** For large venues (e.g. convention centers, stadiums), WeUP can obtain 3D data through various methods:
- **LiDAR Scans:** Using LiDAR scanners (such as a terrestrial laser scanner or handheld devices with LiDAR) to collect point clouds of the venue’s interior and exterior. Modern iPhones and iPads themselves can act as basic LiDAR scanners for smaller spaces, but for high fidelity, a professional scan might be done by the venue (or a contractor). LiDAR produces a “point cloud” – millions of points with 3D coordinates (and possibly color) representing surfaces.
- **Photogrammetry:** Alternatively or additionally, a series of high-resolution photographs (from DSLRs or drones) can be processed to generate a 3D mesh of the venue via photogrammetry software. This can be useful for outdoor environments or detailed textures.
- **Architectural Models:** In some cases, venues might provide CAD/BIM models or floor plans. Those can be converted to 3D meshes. If available, BIM (Building Information Modeling) files contain structured geometry of the building which can be directly leveraged. WeUP’s pipeline can combine these sources – for instance, use a known floorplan layout for accuracy, and drape textures from photogrammetry for realism.
- **Point Cloud Processing:** Raw point clouds from LiDAR need processing before they’re useful in an interactive map:
 - **Cleaning and Filtering:** Remove noise and outliers (erroneous points due to reflections, moving people captured in scan, etc.). Also possibly subsample if density is excessively high beyond what’s needed.
 - **Segmentation:** Depending on needs, the point cloud can be segmented into logical structures (walls, floors, obstacles) especially if we plan to generate a navigable map or if we want to overlay information by semantic area. Some pipelines use plane detection to identify walls/floors in indoor scans ²⁶ ²⁷ .
 - **Meshing:** For visualization, it’s often efficient to convert point clouds into a mesh (triangular surfaces) or Voxels. Meshing can produce a continuous surface model which looks better for things like walls and floors (point clouds can appear noisy or sparse unless very dense). Tools like Autodesk Recap or open libraries (e.g. PDAL followed by Poisson surface reconstruction) can be used.
 - **Tiling and Level of Detail:** Given potentially huge data sizes, we prepare the data in a **tiled, multi-LOD format**. The industry standard approach is Cesium’s **3D Tiling Pipeline**. WeUP can either use

Cesium ion cloud service or a similar tiling tool to convert the cleaned point cloud/mesh into 3D Tiles format. Cesium's tiling will produce an octree or quadtree of tiles with various details ⁸. Each tile contains a portion of the model, and for each area, multiple LOD versions exist (so the client can load low-detail tile when far and swap in high-detail when near). This preserves performance by **streaming only needed data** ⁹. Notably, Cesium's pipeline also **preserves point metadata** if needed ²⁸ – for example, point classifications or colors, which could be useful for analysis (though for pure visualization we mostly care about geometry and texture).

- **Data Storage:** The processed 3D model/tileset will be stored on a cloud storage (like AWS S3 or Azure Blob), preferably a CDN for quick delivery to clients. The Map Service keeps a reference to the asset (e.g. a URL to the tileset JSON, which describes how to load the 3D Tiles). If multiple versions are needed (e.g. a lightweight “floorplan only” version vs a full detailed scan), those can be separate assets. WeUP might maintain both:
- A **simplified model** (low poly count, maybe just critical structures) for lower-end devices or initial loading.
- A **full detailed model** for users who opt in to see high realism (at cost of more data). The decision can be dynamic based on device capability or user setting.
- **Integration with CesiumJS:** On the frontend, CesiumJS will load the 3D tileset asset. This is straightforward once the tileset is hosted – just provide the URL to Cesium's `Cesium3DTileset` API, and it will handle streaming the content as the user navigates ²⁹. We need to ensure the coordinate system is properly georeferenced. Typically, indoor models are placed at the correct latitude/longitude with the correct rotation to align with the real world. The tiling process will embed that transform. If working with a local coordinate (like a building's own coordinates), we'll apply a geospatial transform so Cesium knows where to place it on the globe.
- **Runtime Performance Considerations:** We must optimize the content for real-time use:
 - Too many small tiles can cause overhead; too few large tiles can cause slow loads. We'll configure tiling to find a balance (Cesium ion usually handles this well automatically).
 - We will also consider using **glTF** with Draco compression for mesh data to drastically reduce file sizes for download.
 - For point clouds, using 3D Tiles' **point cloud** type (which streams binary .pnts files) allows even billions of points to be rendered by loading subsets. The Cesium renderer can handle huge point clouds by thinning out distant points.
 - The platform might support toggling certain layers – e.g. users could turn on “architecture view” which shows walls and layout clearly, vs “photorealistic view”. This can be done by having multiple tilesets or styling rules to adjust appearance.
- **Updating and Versioning Models:** Venues can change (booths move, new mapping done). WeUP's pipeline should allow updates. If a venue is scanned again or updated (say, different booth layouts for a new event), a new tileset can be generated. Cesium 3D Tiles supports **refining** and versioning – but simplest is to treat it as a new asset and have the Map Service point to the correct one per event.

We might also break down models per logical area (e.g. separate tilesets for each hall) to make it easier to update small sections.

- Additionally, dynamic objects (like moveable assets or temporary structures) might not be baked into the base model; instead they could be separate layers or represented as standard entities (so that they can be shown/hidden or moved easily). For instance, for a trade show, the base hall is static, but booths might be added as separate models or even simple boxes in the engine, so that each event can reconfigure booth placement without needing a whole new scan (if precise geometry isn't required, we can use parametric booth shapes from data).

- **Processing Tools and Automation:** WeUP will likely automate the pipeline as much as possible:

- Use scripts or cloud functions to ingest raw scan data and run through tiling (maybe via Cesium ion's API or an open-source tiler like **3d-tiles-tools**).
- Ensure quality by reviewing the output model in a sandbox before publishing to the live app.
- Use **metadata**: If scanning via LiDAR, the resulting model might include point classifications (ground, wall) and other metadata. Cesium's pipeline preserves LAS metadata into 3D Tiles for analysis ³⁰ ³¹. In future, WeUP could leverage that metadata for smarter features (like distinguishing between structural elements and props).

In summary, the LiDAR/point cloud pipeline takes the real world and transforms it into WeUP's digital world. By using cutting-edge 3D tiling technology, WeUP can present detailed, realistic venue models to users without overwhelming their devices. This not only wows users with a cool 3D experience, but it provides the spatial context needed for navigation and a sense of presence – it makes a virtual attendee feel like they're "at" the event even if they're interacting through the app.

Event and User Data Graph

WeUP isn't just an isolated event app – it fosters a community and generates rich data from user interactions. At the core of leveraging this is the concept of an **Event and User Data Graph**: a connected data model that links users, events, sessions, shops, social relationships, and activities. Modeling the platform's data as a graph enables powerful features such as personalized recommendations, social network features, and creator analytics. Here's how WeUP approaches the data graph:

- **Graph Data Model:** Each entity in the WeUP ecosystem (users, events, booths/shops, purchases, check-ins, etc.) can be seen as a node in a graph, with relationships (edges) connecting them. For example:
 - A user **attends** an event.
 - A user **follows** another user (social friend or following an influencer).
 - A user **purchases** an item from a shop.
 - A user **likes** or **favorites** a session or booth.
 - A creator (event organizer) **hosts** an event.

- Two users **invite** relationship (who invited whom into the platform).

By capturing these relationships, we can answer complex questions (like "who might be interested in this upcoming event?" or "what items are often bought together at events similar to this one?") more naturally than with purely relational tables. While PostgreSQL can manage relationships via joins, a

dedicated **graph database** or graph processing layer might be employed for recommendation algorithms. In fact, graph databases like Neo4j are known to excel at handling interconnected data and queries about relationships ³². WeUP may use Neo4j or an equivalent for building the recommendation engine and social features, while still using PostgreSQL for core transactional data (the two can be kept in sync through event feeds).

- **Personalized Event Recommendations:** With many events on the platform (and future growth), users will rely on recommendations to find relevant experiences. The recommendation engine will use the graph data:
- **Collaborative filtering approach:** “Users similar to you attended X, you might like Y.” Similarity can be defined via the graph: if two users have attended many of the same events or have many common connections, they’re more likely to share interests ³³ ³⁴.
- **Content-based approach:** Match user’s profile (interests, past events, perhaps explicit preferences they set) with event metadata (tags, categories).
- **Social recommendations:** “Your friend is going to Event Z next week, check it out.” This leverages follow/invite edges.
- **Temporal and contextual factors:** Only recommend events that are upcoming (no point in past events except as archival), and possibly those geographically relevant (if location known).
- The system will likely use a hybrid of these methods ³⁵, possibly employing machine learning (Graph neural networks or simply weighted scoring). Real-time context (time of day, trending events) can also factor in ³⁶. The graph structure is very helpful for such algorithms, and Neo4j’s query language can directly find friends-of-friends attending events etc.
- **Shop and Content Recommendations:** Similarly, the platform can recommend items or booths to attendees. For example, “People who attended the Art Expo and liked the 3D Printing demo also visited the VR Art Booth.” This is analogous to e-commerce recommendations (“users who bought X also bought Y”), which can be powered by analyzing the purchase graph. If using a graph database, one could query patterns like: find other users with similar purchase history and see what they bought that the current user hasn’t. Or use a more straightforward approach with data warehousing and collaborative filtering via matrix factorization. The specifics can evolve, but the data graph ensures we have the relational information captured to fuel these systems.
- **Creator Analytics:** Event organizers and content creators using WeUP will get insights through analytics dashboards. The data graph provides the backend for many metrics:
- **Attendance and Engagement:** How many users attended (virtually or physically)? Which user segments (by tier, interest) were most present?
- **Heatmaps & Movement:** Using aggregated location data, show which areas of the venue were most visited (e.g. a conference organizer sees that 80% of attendees visited the main hall, but only 20% went to the 3rd floor workshops, indicating maybe better signage or content is needed upstairs). This comes from spatial data collection.
- **Shop Sales Analytics:** Vendors see which products sold best, at what times, and possibly correlate with user demographics (e.g. VIP tier spent more on average). This comes from the purchase edges in the data graph.

- **Social Reach:** Because invites are part of the model, an organizer can see how the word-of-mouth spread – e.g. one influencer sent invites that brought 50 new attendees. The “user invite tree” is a particularly powerful representation ³⁷ : it can track how an event’s community grows and who the key connectors are. If someone invited many active users, they could be identified as an ambassador.
- These analytics will likely be delivered by aggregating data in a data warehouse or using graph algorithms. For instance, identifying top influencers could involve PageRank or centrality measures on the invite/follow graph. WeUP could incorporate such algorithms to reward or highlight community builders.
- **Social Graph Features:** WeUP’s invite-only tiered community means the social graph is integral. The platform will likely include:
 - **Friends/Followers:** Users can connect and see each other’s event activities (with permission). This encourages networking – e.g. knowing which events your friends are attending.
 - **Communities or Circles:** Possibly interest-based groups or event-specific communities (like a chat or forum for attendees of Event X). The membership of these can also be considered in the graph.
 - **Gamification via Graph:** As mentioned, the invite graph is used in gamification – e.g. referral leaderboards. Also, users could get badges for attending certain series of events (like completing a “circuit”), which effectively uses the event attendance edges.
- **Moderation and Safety:** A graph helps in content moderation too – e.g. if a user is reported for bad behavior, knowing their connections can help in tracing issues (the user’s tree of invites might be examined if needed to see if a cluster of problematic accounts came from one inviter, for instance, to remove bad actors systematically).
- **Technology Implementation:** The graph may be implemented via a separate service. For example, a **Recommendation Service** might contain a Neo4j instance or use a distributed graph processing engine. It would ingest events from other microservices (like “user attended event” or “user bought item”) and update the graph. This service then exposes APIs: e.g. `GET /recommendations?userId=123` which runs the necessary queries or ML models and returns recommended events/items. Another approach is to use cloud-based analytics tools (like Amazon Neptune for graph DB, or even running algorithms on data in BigQuery or Spark if using more ML). Early on, a simpler approach can be implemented (like daily offline generation of top-N recommendations per user) if real-time is not critical. As the platform grows, more real-time personalized suggestions can be generated on the fly using the latest data.
- **Scalability and Privacy in Data Graph:** Graph queries can become expensive as data grows. We’ll ensure the architecture can handle it by:
 - Offloading heavy computation to offline batch jobs where possible (e.g. computing global recommendations nightly).
 - Using indices and caching in the graph database for frequent queries (like “friends attending this event”).
 - For privacy, ensure that only appropriate data is exposed: e.g. a user can only see recommendations or social info that they are entitled to (their own data, or aggregated community data, not someone else’s personal history without consent). Personalization algorithms should also be scrutinized for

bias or fairness, especially if tiered access is involved (ensure recommendations aren't inadvertently favoring one group unfairly).

In essence, the Event/User Data Graph is the **brain** behind WeUP's intelligent features. It transforms raw logs and relationships into meaningful knowledge: driving the recommendation carousels you see in the app, the "insights" that organizers get post-event, and the connective tissue of the invite-only network. By structuring data as a graph and using the right tools, WeUP can unlock value that sets it apart from a basic event app.

Backend Microservices and Modular Services

WeUP's backend follows a microservice architecture, with each service encapsulating a specific domain of functionality. Below we outline the major microservices (and some smaller supporting services), describing their responsibilities, technology choices, and how they interact. This modular approach ensures the system is scalable and that teams can develop and deploy services independently when needed ³⁸ ³⁹ .

- **User Identity Service:**

Responsible for **authentication, authorization, and user profiles**. Built on .NET 8 (likely using ASP.NET Identity or a custom implementation with JWT tokens). It manages user accounts, including the invite code mechanism and tier information (e.g. a field for user tier or roles like "VIP", "Organizer"). It issues OAuth2/JWT tokens for authenticated sessions, which include claims for the user's roles/tiers for other services to consume ⁴⁰ . It also handles password storage (hashed), social login if applicable, and account linking. The invite system is implemented here: invites can be entities that link an inviter and a code, and upon successful use, the new account is created and a graph edge (inviter->invitee) is recorded (possibly passed to the Graph service). This service will also implement **tier-based access controls** – e.g. only users with tier "Organizer" can create events or only "VIP" can access certain features. These rules are enforced via token claims and also by the gateway and other services trusting those claims.

- **Event Management Service:**

Handles all things related to events themselves (the metadata and configuration of events). This includes creating an event (by organizers), defining the venue (link to map data or venue ID), schedule (sessions, talks), and any settings (public vs private, capacity, ticket prices if any, etc.). It exposes APIs to get event details, search events by criteria, etc. It likely works closely with the Ticketing Service for events that require tickets. In the data model, it stores what users are marked as attending or interested (though these relationships might also be partly in the Graph DB for recommendations). For performance, it might maintain counts of attendees, etc. Because events are core, this service will be high-traffic (many reads). Caching via Redis will be used to cache popular event info (especially during an event, many calls to get schedule or participants might happen).

- **Ticketing Service:**

If events require ticketing (e.g. paid tickets, check-in QR codes), this service manages that. It issues digital tickets or passes, possibly generating QR codes or unique IDs that attendees use to check in at the venue gates. Built with strong consistency in mind (no duplicate ticket IDs, track usage). This service interacts with Payment Service if tickets are sold (charging for ticket purchase) and with the Identity service (assigning tickets to user accounts). It also provides validation endpoints – e.g. a staff scanning a QR code triggers a call to Ticketing Service to mark that ticket as used/checked-in. Given

future blockchain integration goals, this service could integrate with a blockchain module to record ticket issuance on-chain for transparency (see Forward-Looking section for more). But out-of-the-box, it will likely use a database table of tickets, statuses, etc. This service ensures invite-only rules can be enforced too – e.g. only those invited can receive tickets, or perhaps special events where the ticket itself is an invite mechanism (like one invite = one ticket).

- **Map Service:**

Dedicated to serving map-related data. It provides endpoints for retrieving venue maps, floor plans, and related geospatial queries (like “what venues are near this location” if needed). One key function is to serve the **3D Tiles/Map data** – this might not be via a traditional JSON API but rather by hosting the static tile content on a CDN. The Map Service might provide the lookup: given an event ID, it returns the URL(s) of the 3D tileset and any style configuration needed for the front end. It could also offer an API to get points of interest (POIs) within a venue (booth locations, labels) so that the frontend can label the map or offer search (“Find Booth #12”). Another function is geospatial computations: e.g. convert a lat/long to a venue coordinate, or calculate distances. WeUP might incorporate a spatial database (PostGIS) here or use libraries for geometry. Real-time integration: the Map Service might subscribe to updates like crowd positions and provide aggregated data to clients (though that might also be handled directly via the Real-Time service publishing to WebSockets). This service ensures all map data is consistent and easily accessible to the clients.

- **Shop Service:**

Manages shops (vendor booths) and their product listings. Vendors or organizers use this to create a shop, add items (name, description, price, etc.), set inventory counts or link to an Inventory Service. It provides APIs for clients to list all shops at an event, get details of an item, etc. The **Inventory Service** can be considered part of this domain or a tightly coupled sub-service. For scaling, inventory operations (which are highly concurrent) might be separated in their own service or at least in their own DB table with appropriate locks. The Shop Service coordinates with Payment and Order services to execute purchases. It also might integrate with a **Notification Service** to send order updates (e.g. “Your order is ready for pickup at Booth A”).

- **Order/Payment Service:**

It’s possible to combine Order and Payment or keep them separate. The **Order Service** orchestrates the purchase flow, as discussed, and stores order history. The **Payment Service** integrates with external gateways (Stripe, etc.) and ensures secure processing. This Payment Service will store minimal data – mainly records of transactions (maybe last 4 of card, status, but not full card info except tokens). It will also be responsible for handling refunds or payment issues. By isolating payments, the system better secures financial logic and makes compliance (like PCI) easier by minimizing how much of the system touches card data. This service could also handle payouts to vendors if needed (like aggregating sales to pay the vendor after the event, though that might be more of a business process outside the core platform).

- **Real-Time Communication Service:**

While each service could push events to WebSockets, often it’s convenient to have a unified real-time gateway (especially using SignalR, which can use a backplane like Redis or Azure SignalR service to scale). The Real-Time service runs the SignalR hubs that clients connect to. Other microservices send messages to this service when something happens that should be pushed out. For example, the Inventory/Shop service would send a message “item X stock updated” to Real-Time service, which

then broadcasts to all clients in group “item X”. Similarly, the Map/Crowd tracking might publish updates to a “heatmap” channel. If using Azure SignalR, other services can call its REST API to broadcast messages. Alternatively, we use a pub/sub (Redis or RabbitMQ) and the Real-Time nodes subscribe and then forward to websockets. This service must maintain **scalability** for potentially thousands of concurrent connections (big events). Using the cloud SignalR service or a well-architected backplane is important for scaling out horizontally. Security: the Real-Time channel subscriptions will be authorized – e.g. one cannot subscribe to a private event’s channel without permission. SignalR supports passing auth tokens and restricting groups.

- **Gamification Service:**

This handles points, badges, and any game logic (like quests or challenges). For instance, define a “quest” such as “Visit 5 different booths and get 50 points”. The Gamification service tracks user progress, awards points, and triggers badge unlocks. It might integrate with other services by listening to events (like “user visited booth” events from Map or “user made purchase” from Shop) – those events feed into rules that then result in a reward. This service will maintain a point ledger per user, badge records, and possibly leaderboards. Leaderboards can be computed on the fly (sorted queries) or cached. For fairness and integrity, if points could translate to real rewards, careful record-keeping is needed (could even be on blockchain in future to prevent tampering – see forward-looking). Initially, a simple relational model in Postgres will suffice. This service also may feed the front-end with the user’s current level, achievements, etc., and allow the front-end to query “what challenges are available” so it can guide the user.

- **Analytics/Data Service:**

In addition to on-demand queries, there might be a service dedicated to analytics, especially for organizers. This could overlap with Graph/Recommendation service or be separate. It might pre-compute certain metrics and serve them via APIs. For example, after an event, the organizer’s dashboard (which might be a separate web app) calls the Analytics service to get stats (attendance count, heatmap data, sales summary). This service would gather data from others (or from a data warehouse). It might also handle real-time monitoring during events (like a live dashboard of current attendance). In early phases, this might not be a separate service (the needed data can be fetched directly from other services or databases), but as the platform grows, a separate analytics pipeline (with streams and batch jobs) will become valuable.

- **Infrastructure Services (Supportive):**

There are often additional services or components:

- **API Gateway** (as discussed in architecture) – not a microservice per se, but an essential piece, possibly a dedicated container running Ocelot or Envoy, handling request routing and auth verification globally.
- **Logging & Monitoring Service:** Centralized logging (ELK stack or Azure Monitor) and maybe a service to collect and expose application metrics (Prometheus scraping, etc.). This ensures all microservices can be observed for performance and errors.
- **Notification/Email Service:** For sending emails or push notifications (e.g. when an invite is sent, or order confirmation, or reminders for events). Could use third-party like SendGrid via a microservice wrapper that other services call.

- **Content/Media Service:** If users or organizers upload media (e.g. event banner images, product photos for shops), a service or at least an S3 bucket is needed to store these. A simple service to generate signed URLs for upload/download might exist.

- **Communication Between Services:**

As enumerated, microservices use a mix of synchronous and asynchronous communication. For example, when a user buys something:

- Frontend calls Shop/Order Service.
 - Order Service calls Inventory Service (sync) to reserve stock, then Payment (sync or redirect).
 - Once paid, Order Service emits an event “OrderCompleted”.
- Inventory Service, Shop Service, Analytics Service, Gamification Service might all listen to “OrderCompleted” events to do their parts (decrement stock final, increment sales count, update user points, etc.). This decoupling via events means adding a new reaction (like gamification giving points for purchases) doesn’t require changing the Order Service logic – just add a new event handler in Gamification service. WeUP will implement an event bus (like RabbitMQ or Azure Service Bus topics) to facilitate this publish-subscribe model ³. Each service has its own **message queue subscriptions** relevant to it. We’ll design events carefully with schemas that include necessary info but aren’t too large (maybe an event just provides an ID and others fetch details if needed to avoid very large messages).

- **Microservice Tech Stack:**

All these services are built with **.NET 8**, benefiting from the latest performance improvements like better routing and native AOT for faster startups ⁴¹ ⁴². We’ll use ASP.NET Core minimal APIs or controllers depending on each service. Data access via Entity Framework Core or Dapper will be used for Postgres. For inter-service comm, lightweight protocols like gRPC might be used especially for internal comm where performance matters (e.g. a synchronous call Inventory->Payment could be gRPC). The .NET 8 release includes enhancements like better containerization support and performance optimizations particularly useful in microservices (e.g. improved JIT, profile-guided optimizations) which helps us keep latency low ⁴³ ⁴². Each service will be containerized and deployed in a container orchestrator as mentioned.

- **Security and Configuration:**

Each microservice will validate authorization on relevant endpoints – using JWT tokens and checking claims (for example, the Event Service’s “create event” endpoint requires a token with Organizer role). Communication between services (server-to-server) might use internal authentication as well (like mutual TLS or simply shared secrets, since it’s within a secure environment). All secrets (DB credentials, API keys for third-party services like payment gateways) will be managed via a secure config system (e.g. Kubernetes secrets or Azure Key Vault). The microservices should also handle failures gracefully: if Payment service is down, Order service might queue the request or at least return a user-friendly error. Circuit breaker patterns can be applied for resilience in inter-service calls.

In essence, WeUP’s backend is a constellation of focused services that work in concert. This modular design allows the platform to scale features independently and maintain **robustness** – an issue in one service (e.g. Shop) should not bring down the others, and each can be updated or scaled without a full system redeploy.

It also makes the platform more flexible for future changes, as new microservices can be added (for new features like AI bots or blockchain integration) without a major refactor of a monolith.

Authentication, Authorization, and Invite Model

WeUP's user model is **invite-only and tiered**, which introduces special requirements for the authentication and authorization system. This section details how users authenticate, how invites and tiers are handled securely, and how access control is enforced across the platform:

- **User Accounts and Authentication:**

Users will typically register via an invite link or code. When an invite is accepted, the user sets up an account (providing email, password, etc., or using an SSO if we allow e.g. Google login – but still the invite would be required to proceed). WeUP uses **JWT-based auth** for the SPA and mobile clients: once a user logs in, they receive a JWT access token (and refresh token if using that flow). This token is required for all API calls and includes the user's unique ID and roles/tiers in its claims. The Identity Service uses ASP.NET Core Identity under the hood for managing credentials and can integrate with external identity providers if needed (for example, enterprise partners might want SAML SSO for their staff – this could be an extension later). Passwords are hashed (e.g. using bcrypt or Argon2) and multi-factor authentication (2FA) is an option for account security (especially for organizer accounts or high-tier accounts).

- **Invite-Only Registration:**

The invite system will likely work as follows: an existing user (with invite privileges) can generate an invite code or send an invite email to someone. The invite is represented in the database with attributes like who invited, how many uses allowed (sometimes invites are single-use, or an organizer might have a bulk invite link for many attendees), and an expiration. When a new user comes in with that code, the Identity service verifies it and allows registration. After registration, the code is marked as used (or decremented if multi-use). The system also establishes the relationship between the inviter and invitee – e.g., storing an “InvitedBy” field on the new user or a separate table mapping both. This essentially builds the **“user's tree” of invites** ³⁷. This tree is important as a social structure: WeUP can display to users some non-personally-identifiable info like “You were invited by Alice, who was invited by Bob” possibly, and for moderation, as themekraft pointed out, it gives transparency and control (if a certain branch of invites causes trouble, admins can prune it) ³⁷. To incentivize invites, WeUP may give a limited number of invites per user or per tier (e.g. a new user might after some engagement earn the ability to invite 3 others). Gamification ties in here as well – inviting might yield reward points or badges, fostering growth via referrals ⁴⁴.

The invite codes themselves should be secure (random enough to not be guessable) and short enough to relay (maybe 8-10 chars). Alternatively, invite links containing a token are used. We will protect against invite code reuse if not allowed and ensure invites cannot be generated by unauthorized users.

- **Tiered User Roles:**

WeUP's community is tiered/invite-only, which implies multiple roles or levels:

- **Standard Member:** A normal invited user.
- **VIP Member:** Possibly users who have special status – maybe they pay for a premium subscription or earned it through contributions. They might get access to exclusive events or features.

- **Event Organizer / Creator:** Users who can create events on the platform. They may have admin rights over their event's data.
- **Vendor:** If shop owners are separate from normal attendees, they could have accounts that mark them as booth managers to use the shop management features.

- **Administrator:** Internal staff or system admins with full access.

These roles are implemented in the Identity service and embedded in JWT claims (like a “roles” claim or specific tier claim). The API Gateway and each microservice will check these claims to authorize actions. For example, the Ticketing service will allow only an Organizer (of that event) or an Admin to view the list of attendees for an event. The Shop service might ensure only a Vendor associated with a shop or an Admin can add products to that shop. The **invite distribution** might also depend on tier: e.g. VIP members might have more invite slots or be allowed to invite directly to VIP tier (just speculation, but possible).

- **Access Control Enforcement:**

We follow the principle of **least privilege**. By default, APIs require a valid user token. Specific endpoints check for roles:

- e.g. `POST /events` requires role Organizer or Admin.
- `GET /events/{id}/analytics` requires Organizer of that event or Admin.
- `POST /shops/{id}/items` requires that the current user is the owner of that shop or an event admin.

These checks will be done via attributes or middleware in .NET (policy-based authorization). Additionally, within functions, important business rules double-check (defense in depth). The Real-Time service also will restrict subscriptions. For instance, a user can only join a SignalR group for an event if they are marked as attending that event (or events could have public data channels that anyone at the venue can join, but private chats would be locked down). The system should also guard against **horizontal privilege escalation** – user A should not access user B's data. Resource-based checks (like verifying the owner of a resource matches user ID from token) are needed. For example, if an invite code is tied to an inviter ID, only that inviter or an admin should fetch details about it.

- **Session Management & Device Trust:**

Users might use multiple devices. WeUP will allow that but might have a concept of device management (maybe a user can see what devices are logged in and sign them out, for security). We might also want to detect anomalies (account sharing or theft – if one account logs in from two far apart locations simultaneously, could flag). However, initial version likely keeps it simple with token expiration and refresh.

- **Invite Links Security:**

An invite link may grant access to registration, but once used, it shouldn't allow someone else to reuse it. We'll ensure one-time links or code invalidation after use. Also, if an invite link is posted publicly by accident, an admin might want to revoke it, so we'll have an interface to deactivate invite codes.

- **Community Management and Moderation:**

The invite-only model tends to foster a well-behaved community, as noted by themekraft: since

people invite those they know, a baseline trust or homogeneity can make the environment positive ⁴⁵. But if someone misbehaves (posting inappropriate content in event chats, harassment, etc.), moderation tools are needed. Admins can trace through the invite tree to see who brought that person ³⁷, and possibly take action such as warning the inviter if multiple invitees cause problems – this approach can encourage people to invite responsibly ⁴⁶. This is a unique dynamic in invite communities: people have a stake in who they invite because it can reflect on them. The platform might automate a bit of this (for instance, temporarily disabling invite privileges for a user whose invitee was banned for cause, etc., though that requires careful policy to be fair). Community guidelines will be published, and transparency given that it's not a free-for-all public network means generally higher quality discourse.

- **Future Integration (SSO, etc.):**

While initial focus is on invite codes, in the future WeUP might partner with organizations that want to use their own identity. The architecture can accommodate federated identity (e.g. OAuth social logins or SAML for enterprise). But even in those cases, an invite or some gating mechanism would still likely apply (like you can sign in with Google, but only if your email is on an invited list or you have an invite token, etc.).

- **Data Security:**

All communication will be over HTTPS/TLS. Passwords never emailed in plain text, etc. The identity data (like password hashes, emails) will be in a secure DB. We will likely implement rate limiting on auth endpoints to prevent brute force and multi-factor for admin accounts. The platform should also consider GDPR or similar compliance since user personal data is involved – provide means to delete an account and associated data if requested.

In conclusion, the Auth system of WeUP not only secures access but also embodies the exclusive community philosophy. By designing a robust invite flow and role system, WeUP ensures that only the right people get in, and once in, each user's permissions are appropriate to their role/tier. This creates a sense of trust and exclusivity which is core to the platform's appeal, while still maintaining scalability and security akin to any modern web platform.

Real-Time Infrastructure and Updates

Real-time capabilities are critical in WeUP to deliver instantaneous updates and interactive experiences. From live map and shop updates to chat and notifications, the platform's real-time infrastructure needs to be robust and scalable. Here we detail how WeUP handles real-time data flows, including the use of WebSockets, managing high-frequency events like user movement, and handling peak scenarios like flash sales with fairness:

- **WebSocket Communication (SignalR):**

WeUP uses WebSockets for bi-directional, low-latency communication between clients and server. Specifically, as a .NET-based system, **ASP.NET Core SignalR** is the chosen framework. SignalR provides an abstraction over WebSockets (falling back to other transports if needed) and allows servers to push messages to connected clients easily ⁴ ⁴⁷. We will define various **SignalR hubs** or channels for different real-time features:

- **A Map Hub** for pushing map-related events (crowd updates, friend locations, etc.).
- **A Shop Hub** for inventory and flash sale notifications.

- Possibly a **Chat/Notification Hub** for user messages or general notifications.

Each hub can have groups to which messages are targeted (for instance, a group per event for event-wide announcements, or per booth for shop-specific updates). The SignalR service (as discussed earlier in architecture) will run on multiple servers; we'll use the built-in scale-out via Redis or Azure SignalR which handles routing messages to users regardless of which server they connected to. The key benefit is user experiences like *"the map updated showing my friend moved a few meters"* or *"an alert popped up that a session is starting now"* happen in real time without user refresh.

- **User Movement and Map Updates:**

If allowed, WeUP can broadcast some user movements on the map. However, broadcasting every single user's live GPS coordinate to every other user doesn't scale (and is a privacy issue). Instead, we handle movement in a controlled way:

- **Friends/Group Sharing:** Users can explicitly share location with friends. In that case, the app will send the user's location to the server (perhaps via a throttle, e.g. every 2 seconds when moving). The server then broadcasts that update only to the friend(s) or to that user's "party" group on SignalR. That way, small groups can see each other moving in near-real-time (a small number of subscriptions).
- **Crowd Heatmap:** For general crowd info, the app might periodically send anonymous location pings to the server (which aggregates them). Instead of broadcasting individual positions, the server computes a heatmap data structure (e.g. a grid of counts or clusters) and broadcasts an updated heatmap maybe every 30 seconds to all clients in that event. This significantly reduces data – clients get an abstract of crowd density, not raw points. The map can then visualize it. This approach informs users of crowd flow without revealing individuals, and scales to thousands of users by condensing data.
- **Session/Booth Presence:** WeUP can also do specific updates like "X number of people are at Booth 12 right now". This could be an aggregate count updated occasionally. The system ensures these updates are efficient by collecting data on backend and pushing only needed info. If using SignalR, we can have a group for each event's heatmap or each booth count. Clients join those groups if they are viewing that context.
- **Live Shop and Inventory Updates:**
Inventory changes (like item stock) should be sent out immediately to interested clients. As discussed, the Shop/Inventory services will trigger events for changes. These will be forwarded to clients via the Shop Hub on SignalR:
- We might have groups per item or per shop. Clients viewing a particular shop page join that shop's group; then "item X stock=0" message is broadcast to that group, so all users there see the item become sold out at once.
- For flash sales or limited offers, we also need to manage concurrency. WeUP could implement a **queue system** where, say, at 3PM a flash sale opens, but only 100 people can buy an item. Instead of letting all 1000 interested users hit "Buy" at once (which could overwhelm inventory and payment), we may use a queue:

- Users click to join the sale queue. The system assigns positions (1 to N).

- The Real-Time service can update each user with their position and updates (“Now serving #1-50”).
- As transactions complete, the next users are allowed to purchase. This is akin to online ticketing waiting rooms. It ensures fairness and that inventory count remains correct by throttling the purchase attempts.

While such a system is complex, building some basic support is feasible. For instance, Redis could hold a list of queued user IDs and a counter for how many have been let in. Real-time feedback to users keeps them engaged (they know they’re in line, not just an error). If they leave, we remove them from queue. This is a niche but important scenario for high demand moments.

- **Scalability and Performance:**

The Real-Time system must be designed to handle thousands of concurrent connections especially at big events or during global live streams. Using Azure SignalR Service (a managed hub that can scale to many connections without us managing the servers) is an attractive option. If self-hosting, we ensure the VM or container can handle the connection count and message throughput – which might mean dedicating certain nodes just for SignalR hubs. The architecture with microservices allows scaling the real-time component separately.

For message throughput, not all users will require high frequency messages. Friend location updates might be 1/s per friend in view. Heatmap updates might be 1 per 30s. Inventory changes are sporadic. This is all manageable. However, we must avoid the “thundering herd” in flash sale or announcement – e.g. broadcasting a notification to 10k users at once. SignalR can generally handle that by sending over established sockets rather than each user polling (which is worse). We will still test and possibly batch messages if needed (like sending a message that clients know to interpret as a general event, etc.).

- **Reliability of Real-Time:**

WebSocket connections can drop (especially on mobile networks). The client app must gracefully handle reconnects – SignalR has automatic reconnect options which we will enable. If a user goes offline for a bit, they might miss some transient updates (like position blips), which is acceptable; but critical events (like “your purchase succeeded”) should also be communicated via other means (e.g. a stored notification that syncs when back online). Real-time is a bonus channel, but the system should not rely solely on it for critical state changes – those should also be persisted or be queryable. For example, if an inventory sold out and a user was offline, when they come back, the API for that item will show 0 stock.

- **Use of Push Notifications:**

Outside of the web socket realm, mobile apps might use push notifications (APNs for iOS, FCM for Android) for certain things like “Your friend sent you an invite” or “Event starting in 10 minutes”. This is asynchronous but not exactly real-time interactive like SignalR. A Notification Service integrated with Apple/Google push can complement SignalR for reaching users who might not have the app open.

- **Example Real-Time Flows:** To illustrate:

- **Map Friend Update:** Alice and Bob are friends. Alice is sharing location. Every few seconds, Alice’s app sends her new coordinates to the backend (Map Service or Real-Time service via some method,

perhaps an HTTP call that then triggers a SignalR message). The server knows Bob is allowed to see Alice (they're friends) and knows which connection corresponds to Bob (because Bob's client subscribed to friend updates). The server sends Bob a message "FriendLocation: {friendId: Alice, lat: ..., lon: ...}". Bob's app updates Alice's icon on the map. Alice moves through the venue and Bob sees it with only a small delay.

- **Inventory Update:** A rare collectible at Booth 9 had quantity 3. Three different users purchase almost simultaneously. The Inventory Service serially processes each: after third purchase, quantity=0. It then sends a message via Real-Time to all in "Booth9" group: "Item123 stock 0 (sold out)". All users looking at that booth instantly see that item as sold out (perhaps the buy button becomes disabled and a "Sold Out" label shows).
- **Flash Sale Start:** At 5PM, a flash sale is scheduled at Shop X. A server-scheduled job or admin triggers a SignalR broadcast to "ShopX_flashsale" group: "Flash sale is live now!". Users get a pop-up. They click purchase – the order requests might be queued but at least everyone got the start signal at the same time (which is more fair than some having to poll and maybe seeing it later).

In summary, the real-time infrastructure ensures WeUP feels **alive and synchronous**, like a real event. Whether it's seeing live activity on a map, grabbing a hot item the moment it's available, or simply receiving instant notifications, this layer is what makes the platform interactive rather than static. By using proven tech like SignalR and careful design around grouping and event broadcasting, WeUP can achieve this interactivity at scale.

Forward-Looking Capabilities and Future Integrations

While the core of WeUP is defined by the features discussed, the platform is designed with extensibility in mind to incorporate emerging technologies. In this section, we outline some forward-looking capabilities that, although not necessarily in the initial launch, are on the roadmap. These include AI-driven features, augmented reality modes, and blockchain integrations for trust and new business models. Embracing these in the architecture early ensures WeUP remains **future-proof** and innovative:

- **AI Agents and Assistants:**
WeUP can leverage Artificial Intelligence in numerous ways to enhance user experience:
- **Virtual Event Assistant:** An AI chatbot integrated into the app that can answer attendees' questions in real-time (like "When is the next keynote?" or "Where is the restroom?"). This could be powered by a large language model (LLM) fine-tuned on the event's data (schedule, venue map) so it can provide context-specific answers. The chatbot might appear in the event info section or as a voice assistant in AR mode.
- **Recommendation Engine AI:** While we discussed recommendations via data algorithms, AI can improve this by analyzing richer contexts. For example, an AI could analyze not just your past events but maybe sentiment (from any feedback or social media integration) to tailor suggestions. Over time, a model could be trained for "event affinity" to predict which events a user will like with high accuracy.
- **Content Generation:** Organizers might use AI to help generate event descriptions or summary after-events. For instance, after an event, an AI could automatically generate a highlight reel or a written summary article analyzing participation and sentiment (if we gather feedback surveys or social feeds).
- These AI components would likely run as separate services (for instance, using Azure Cognitive Services or OpenAI APIs). They would require relevant data (the Knowledge Base of event info for QA

bots, or user preference data for recs). WeUP's architecture, with an event graph and data pipeline, ensures data is accessible for training/feeding AI models. We can integrate AI results back into the platform seamlessly (e.g. the chatbot's response is just another real-time message in the chat UI).

- **Augmented Reality (AR) Mode Integration:**

As discussed under positioning, AR offers a compelling way to navigate events. Future versions of WeUP could have a dedicated AR mode where:

- Users hold up their phone and see **directional arrows or indicators** overlaid on the camera view guiding them to a selected destination (e.g. an arrow pointing down the hall with "Booth 5 this way, 30m"). AR wayfinding is a trend for indoor nav ⁴⁸ ⁴⁹ and can greatly reduce confusion in complex venues.
- AR overlays on points of interest: Point your camera at a booth and see a floating label with the booth name and maybe current promotions. Or point at a crowd and see a tag like "Registration line starts here".
- **AR gaming elements:** Gamification can be enriched with AR. For instance, an Easter egg hunt where virtual objects are placed around the venue in AR for users to find (earning points). Apple's ARKit and Google's ARCore can both anchor virtual content to real-world, so WeUP could create AR quests that encourage exploration.
- Another idea: **AR photo ops** – e.g. a user can see a virtual mascot or stage in AR and take a selfie with it, shareable to the community. Implementing AR features means ensuring our 3D map data is AR-friendly. With ARKit's scene understanding, since we already have a 3D model, we could align the AR virtual content with the real world better. The app might download a lightweight AR model of the venue for alignment. Apple's newer APIs like ARGeoAnchors can place content using global coordinates (GPS + visually refine), which might be used outdoors. Indoors, ARKit's LiDAR gives excellent mapping – as noted, ARKit with LiDAR significantly enhances indoor navigation by providing depth and anchors ²¹. By staying updated on AR tech, WeUP can gradually roll out these features on capable devices.

- **Blockchain Modules (Stellar/Soroban):**

WeUP is exploring blockchain, particularly with **Stellar** and its smart contract platform **Soroban**, to bring transparency and new capabilities around transactions:

- **Ticketing on Blockchain:** As referenced by projects like EntryX, issuing event tickets as digital assets on Stellar allows for a more transparent and fraud-resistant system ⁵⁰. Each ticket can be an NFT-like token. Soroban smart contracts manage sales and transfers, enforcing rules (like price caps, limiting resales) automatically ⁵⁰. This means when a user buys a ticket, a token is minted to their wallet, which they truly own. It can be verified publicly, eliminating counterfeit tickets ⁵¹. If they resell it, that transfer is logged on-chain, and rules like royalties to the organizer or preventing scalping can be coded in.
- **Resale and Escrow:** A big problem in events is secondary markets (reselling tickets) – often rife with fraud or exorbitant prices. By using Soroban contracts, WeUP could support a built-in resale marketplace where smart contracts hold escrow (funds and ticket) and swap when conditions meet, ensuring fair trade ⁵¹ ⁵². Soroban can also enforce that resale price doesn't exceed a certain percentage above face value if the event wants to discourage scalping. Or if allowed, at least

everyone can see the history of a ticket (traceability) – who owned it, for how much it was sold ⁵³ . This transparency might deter malicious actors and builds trust with fans and organizers ⁵³ .

- **Merchandise and Digital Collectibles:** Beyond tickets, shops at events could sell NFT-based merchandise (like a limited edition digital art or badge for attending the event). Stellar's low transaction fees make it feasible to use for micro-transactions, and Soroban could handle more complex logic if needed (like combining items, or gamified NFTs).
- **Payments and Currency:** WeUP could integrate cryptocurrency payments or its own token for event economies in the future. For example, a "WeUP token" that attendees earn (from gamification or refunds) and can spend on any event's shops. Stellar is designed for currencies and could facilitate this with fast settlement.
- **Technical Integration:** Introducing blockchain can be done in parallel to existing systems. We might have an optional **Blockchain Service** that if enabled for an event, will mirror certain operations to the Stellar network. For example, when a ticket is sold in WeUP, the Ticketing service calls the Blockchain service to also mint the Stellar asset or call a Soroban contract. This requires users to have or be linked to a Stellar wallet (e.g. via something like Freightier browser wallet ⁵⁴ for web, or we integrate a custodial wallet solution for convenience). Initially, this could be opt-in and behind the scenes. The EntryX example shows requiring the user to approve with their wallet ⁵⁴ , but for mainstream adoption, WeUP might abstract it (the user sees a normal ticket, the platform handles the token behind if they don't care to manage keys).
- **Escrow Smart Contract Example:** The Soroban contract ensures that for auctions or resale, a bid is only valid if the bidder's funds are actually put in escrow on-chain, eliminating fake bids ⁵² . This was exemplified by requiring a token transfer at bid time ⁵⁵ . WeUP could incorporate such robust mechanisms for any scenario of high value exchange in the platform. It's worth noting that these technologies would be carefully security-audited if used, and always have an "off-chain" fallback in case a user doesn't have crypto setup (WeUP might manage a custodial account on their behalf by default).
- **Edge Computing and IoT:** (Additional forward idea) If events deploy IoT devices (like live sensors counting people or interactive kiosks), WeUP could integrate those streams too. For example, camera-based counters giving more accurate crowd counts, or smart signage that updates via our system. These are more infrastructure-heavy ideas but align with a high-tech event platform. Edge computing might be used in huge venues to locally process some data (like massive crowd video analytics) and only send results to cloud.
- **Open APIs for Partners:** In the future, WeUP might expose APIs or SDKs for external partners (like a venue's own app or an exhibitor's system) to integrate. Designing our system with API-first principles now will make it easier to safely expose such endpoints later without internal rework.

Each of these forward-looking modules (AI, AR, blockchain) will be developed as **add-on microservices or client features** that do not disrupt the core. The architecture already supports plugin-like addition: e.g. the blockchain integration can be an optional path in the ticketing or payment workflow; AR features mostly affect the client and use existing data pipelines; AI services consume data but don't change data generation upstream. We ensure core services emit the necessary data (events, logs) that these new features need.

By planning for these evolutions, WeUP can incrementally roll them out and stay ahead of competitors. The result will be a platform that not only serves current needs but is ready to incorporate the **"next big thing"**

in event tech, whether it's an AI that personalizes your entire event schedule or a blockchain-secured ticket that doubles as a collectible. This visionary outlook keeps WeUP's technology stack both **agile and cutting-edge** ⁵⁶, much to the benefit of its users and partners.

Conclusion

In this Technical Requirements Document, we've outlined the comprehensive architecture and design of WeUP – an innovative event platform blending 3D maps, real-time commerce, and community engagement. The proposed solution is **modular**, leveraging a .NET 8 microservices backend to ensure each feature set (mapping, shopping, social, etc.) can scale and evolve independently. The frontend experience is enriched with modern web and mobile technologies (Vue 3 + Quasar) and poised to incorporate AR and other emerging tech for an immersive user journey.

Crucially, WeUP's design emphasizes **real-time interactivity** and **data-driven intelligence**: from live map updates over WebSockets to an event/user graph powering recommendations and analytics. We have chosen proven technologies like Cesium for 3D visualization and SignalR for realtime communication, grounding the platform in solid technical footing while allowing forward-looking extensions (AI integration, blockchain ticketing via Stellar Soroban, etc.) that keep it future-ready. Security and exclusivity are ensured via a robust invite-only authentication model and careful role-based access throughout the system.

This document serves as a blueprint for engineering teams to build and integrate the components of WeUP. Each section can be treated as a set of requirements and guidelines for implementation: - The **3D Map Stack** must deliver seamless indoor-outdoor transitions and dynamic overlays as specified, using the described technologies. - The **Live Shop** must guarantee consistency and real-time feedback in the high-concurrency scenarios outlined. - The **Geolocation** subsystem should be implemented with a hybrid of GPS, BLE, and possible AR techniques to achieve indoor accuracy. - The **Microservices** must be developed with the described responsibilities, APIs, and data stores, keeping in mind the communication patterns and scaling considerations. - The **Auth model** needs to enforce the invite and tier logic at every step, aligning with the community goals of WeUP. - The **Real-Time infrastructure** should be set up to handle the event-driven use cases and peak loads (like flash sales), validating the approach with load testing.

Finally, by anticipating future integrations (AI, AR, blockchain), we ensure that the system's architecture is not a dead-end but rather a foundation on which new capabilities can be layered with minimal friction. This forward-compatible design will allow WeUP to integrate features that keep it at the forefront of event technology trends, offering continuous value to users and partners.

The next steps would include turning these requirements into actionable development plans: defining specific APIs, database schemas, selecting specific cloud services (for deployment, scaling, monitoring), and establishing a timeline for iterative delivery (perhaps starting with core features like maps and shops, then layering on community and gamification, etc.). With this TRD as a guide, the engineering team and partners should have a clear, shared vision of the system we are building – one that is technically sound, user-centric, and truly **next-generation** for event experiences.

¹ Typical Microservice Architecture - ByteByteGo

<https://bytebytego.com/guides/what-does-a-typical-microservice-architecture-look-like/>

2 eShopOnContainers Architecture [7] | Download Scientific Diagram

https://www.researchgate.net/figure/eShopOnContainers-Architecture-7_fig1_371490747

3 16 17 40 Maintaining accurate, real-time inventory synchronization across multiple e-commerce platforms like Shopify, Magento, WooCommerce, and custom marketplaces is critical to preventing overselling, optimizing order processing, and ensuring customer satisfaction. A scalable API architecture that supports seamless order workflows and instant stock updates enables businesses to expand efficiently without operational bottlenecks.

<https://www.zigpoll.com/content/can-you-outline-a-scalable-api-architecture-for-integrating-our-realtime-inventory-system-with-multiple-ecommerce-platforms-to-ensure-seamless-order-processing-and-stock-updates>

4 47 Real-time ASP.NET with SignalR | .NET

<https://dotnet.microsoft.com/en-us/apps/aspnet/signalr>

5 6 7 8 9 13 14 15 29 38 39 Inside Cesium: Scalable 3D Geospatial Architecture for Enterprise Use - Janea Systems

<https://www.janeasystems.com/blog/cesium-architecture-3d-geospatial-platform>

10 11 12 MapsPeople builds with Mapbox

<https://www.mapbox.com/showcase/mapspeople>

18 19 20 21 22 AR Indoor Navigation Application Development Guide 2025

<https://mobidev.biz/blog/augmented-reality-indoor-navigation-app-development>

23 24 25 48 49 A complete guide to developing augmented reality indoor navigation applications (2025) | Volpis

<https://volpis.com/blog/guide-to-developing-augmented-reality-indoor-navigation-applications/>

26 A New Framework for Generating Indoor 3D Digital Models ... - MDPI

<https://www.mdpi.com/2072-4292/16/18/3462>

27 Point cloud processing: 3D indoor scans of buildings as essential ...

https://www.researchgate.net/publication/387230378_Point_cloud_processing_3D_indoor_scans_of_buildings_as_essential_tools_in_engineering_design_and_construction

28 30 31 Preserving More Metadata for Point Clouds Using 3D Tiles – Cesium

<https://cesium.com/blog/2024/03/20/preserving-more-metadata-for-point-clouds-using-3dtiles/>

32 33 34 35 36 56 Recommendation Systems For Event-Based Recommendations

https://www.meegle.com/en_us/topics/recommendation-algorithms/recommendation-systems-for-event-based-recommendations

37 44 45 46 Are Invite Only Communities the Future of Social Networks? - ThemeKraft

<https://themekraft.com/blog/are-invite-only-communities-the-future-of-social-networks/>

41 Building High-Performance Microservices with .NET 8: Patterns and ...

<https://medium.com/@james-joseph/building-high-performance-microservices-with-net-8-patterns-and-pitfalls-2da29027c14d>

42 5 .NET 8 Performance Secrets Every Developer Should Know

<https://itnext.io/5-net-8-performance-secrets-every-developer-should-know-6bc39280f7ea>

43 Building Microservices With .NET 8 - Julio Casal

<https://juliocasal.com/blog/Building-Microservices-With-Dotnet-8>

50 51 52 54 55 Entry•X | Decentralized Ticketing Platform - DEV Community

<https://dev.to/joosetoscana/entryx-decentralized-ticketing-platform-3ak1>

53 Everything Event Planners Need to Know About Blockchain Tickets

<https://www.hytix.com/blog/blockchain-tickets/>